# AI MEDICAL INTELLIGENCE PLATFORM

Production-Grade Chest X-Ray Pneumonia Detection System
incorporating EfficientNetB0, Grad-CAM Explainable AI (XAI), and Gemini LLM reporting.

| | |
|---|---|
| **Candidate Role:** | Senior AI/ML & Full-Stack Engineer |
| **Target Domain:** | Digital Health & Intelligent Radiology Diagnostics |
| **Core Stack:** | FastAPI, React, TypeScript, TensorFlow, SQLAlchemy, SQLite, Docker |
| **Document Version:** | 1.0.0 (Production Release) |
| **Submission Date:** | July 22, 2026 |

# 1. Executive Summary & Objectives

This project reports on the technical design and architectural execution of the MedTool AI Medical Intelligence Platform. The system provides a production-grade, end-to-end intelligent pipeline that accepts frontal chest radiographs, performs deep learning classification to identify features associated with active pulmonary pneumonia, generates a visual heatmap highlighting diagnostic focal areas using Grad-CAM, and orchestrates Gemini API to generate structured clinical reports.

### Key engineering objectives achieved include:

- **Deep Learning Model Excellence:** Fine-tuned an EfficientNetB0 network using transfer learning to classify Normal vs. Pneumonia radiographs.
- **Explainable AI (XAI):** Developed custom Grad-CAM visualization to highlight specific spatial features that influenced network decisions, satisfying crucial clinical explainability constraints.
- **FastAPI REST Server:** Engineered a highly responsive async REST API, exposing endpoints for inference, database logs, and granular status checks.
- **Robust Frontend UI:** Built an interactive React dashboard with Vite and TypeScript, featuring a dual-layer cross-fade slider for original/Grad-CAM overlays, historical log inspection, and PDF report downloads.

# 2. Dataset & Preprocessing Pipeline

The model is developed on the chest radiograph pneumonia dataset, consisting of anterior-posterior (AP) and posterior-anterior (PA) view X-ray scans of pediatric patients. Images are cataloged into NORMAL (no active infiltrates) and PNEUMONIA (visible pulmonary consolidation).

### To ensure zero bias and robust evaluation, we implement the following pipeline steps:

1. **Strict 70:15:15 Split Strategy:** The dataset is partitioned into Training (70%), Validation (15%), and Testing (15%) splits. The test partition is held strictly separate and is never loaded during fitting or callback operations.
2. **Input Matrix Standardization:** Images are dynamically loaded, converted to 3-channel RGB, resized to 224x224 pixels, and normalized by dividing color channel values by 255.0 to map input pixel scalars to a [0.0, 1.0] float range.
3. **Online Augmentation:** During training, images are randomly subjected to horizontal flips and slight intensity adjustments to build structural invariance.

# 3. Deep Learning Model Architecture

We implement **EfficientNetB0** as the primary classifier. It uses compound scaling to balance depth, width, and resolution, yielding superior feature representation with minimal model parameters. The architecture begins with pre-trained ImageNet weights, freezing the convolutional base during initial cycles. A customized classification head is appended:

```
Input (224x224x3) → EfficientNetB0 Base → GlobalAveragePooling2D → Dense (128, ReLU) → Dropout (0.3) → Dense (1, Sigmoid)
```

**Model Versioning & Version Control:** The training script outputs the optimized weights as best_model.keras alongside a structured model_metadata.json. This JSON files acts as a version-control log, tracking metrics (accuracy, recall, precision, F1), training timestamps, hyperparameters, and dataset parameters. **In-Memory Caching:** The FastAPI server imports the model once during lifespan startup. The cached model is held in memory and shared across requests, preventing redundant disk loads.

# 4. Explainable AI (Grad-CAM)

Medical AI systems require auditability. We implement **Gradient-weighted Class Activation Mapping (Grad-CAM)**. Grad-CAM uses the gradients of the target class score flowing into the final convolutional layer of the network (top_activation in EfficientNetB0) to produce a coarse localization map highlighting the regions in the image most relevant for prediction.

### The mathematical process is structured as follows:

1. Extract the feature map activations $A^k$ of the final conv layer and compute gradients of the class output score $y^c$ with respect to the activations.
2. Compute the channel-wise importance weights: $\alpha_k^c = \frac{1}{Z} \sum_{i} \sum_{j} \frac{\partial y^c}{\partial A_{i,j}^k}$.
3. Generate the heatmap via a weighted combination of feature maps filtered through a ReLU activation: $L_{Grad-CAM}^c = ReLU(\sum_{k} \alpha_k^c A^k)$.
4. Resize the heatmap to the original image dimensions, normalize, and overlay using a JET colormap blend.

# 5. LLM Clinical Reporting & Fallback

To compile findings into actionable clinical reports, the API hooks into the **Gemini API** (gemini-1.5-flash) using structured JSON prompts. If the API key is not configured, or if rate limits block the request, the backend automatically transitions to a **deterministic local report generator**. This generator utilizes pre-defined clinical templates based on the predicted class and confidence score, guaranteeing full pipeline reliability without breaking.

**Confidence Threshold Constraint:** The application supports a configurable confidence threshold (default 70%). If a prediction falls below this setting, the report is annotated with a warning flagging it as a 'Low Confidence Prediction' and advising immediate physician verification.

# 6. REST API Design & Database Schema

The backend is developed with **FastAPI**, offering asynchronous non-blocking routes and auto-generated Swagger UI (/docs) and ReDoc (/redoc). Heavy operations (predict, Grad-CAM, report formatting) are offloaded to Python's background threadpools to avoid blocking the event loop.

| Method | Endpoint | Description | Status Codes |
|---|---|---|---|
| POST | /predict | Validate upload size/type, run inference, generate Grad-CAM, persist log to DB | 201 Created, 400 Bad Input, 503 Unavailable |
| GET | /history | Query SQLite prediction records (supports search & sorting) | 200 OK, 500 DB Error |
| GET | /history/{id} | Fetch complete prediction and report details | 200 OK, 404 Not Found |

| | | | |
|---|---|---|---|
| DELETE | /history/{id} | Delete prediction record and purge images from filesystem | 204 No Content, 404 Not Found |
| GET | /health | System health report (DB connection, model cache status, LLM availability) | 200 OK |

**Database Abstraction:** We use SQLAlchemy ORM. The default database is SQLite for easy evaluation. However, the database layer is written database-agnostically, allowing quick PostgreSQL integration via environment variable swaps.

# 7. Frontend UI Design

The frontend is constructed using **React (Vite + TypeScript + Material UI)**. The user experience is prioritized, featuring glassmorphic designs, micro-animations, and loading progress state indicators. Key pages include:

- **Dashboard:** Renders aggregated metrics (total scans, diagnostic ratios, average confidence) fetched dynamically from the database.
- **Analyze (Upload Workspace):** Implements client-side checks (JPG/PNG only, <10MB), file drops, and progress spinners.
- **Interactive Grad-CAM Slider:** A double-layer image visualizer with a smooth slider overlay. Users can cross-fade between the anatomical X-ray structures and the model activation centers.
- **Report & PDF Export:** Renders structured clinical summaries and supports client-side PDF downloads utilizing jsPDF.

# 8. Deployment & Containerization

For production hosting, we provide a multi-stage Dockerfile and docker-compose.yml. The multi-stage build compile the React assets using Node, copies them directly to the Python runtime, and mounts them as static files served by FastAPI. This results in a single, high-performance container that starts without manual configuration.

The container incorporates a **Docker Health Check** that queries the FastAPI /api/health endpoint. Persistent data volumes are mapped for SQLite database logging, uploaded original X-rays, Grad-CAM results, and rotating application logs (stored in logs/application.log).

# 9. System Limitations & Future Work

**Limitations:** Currently, the platform relies on binary chest X-ray classification. In clinical practice, patient presentation is often complex, involving co-occurring pathologies (e.g. pleural effusion, atelectasis, cardiomegaly).

### Future Enhancements:

1. **Multi-Label Classification:** Transition the EfficientNetB0 classifier to a multi-label classification head using sigmoid activations across 14 target classes.
2. **Saliency Mapping Comparison:** Integrate secondary explainability models (Integrated Gradients or LIME) to complement Grad-CAM.
3. **PACS / DICOM Support:** Integrate Orthanc DICOM server connectivity to let clinicians query scans directly from their picture archiving and communication systems.

# 10. Conclusion & Legal Disclaimer

The MedTool AI Medical Intelligence Platform demonstrates a complete, production-ready full-stack software system. By packaging complex ML pipelines, explainability models, and generative AI reporting inside a robust web interface, the application bridges the gap between deep learning research and clinical workflow support.

**Legal Disclaimer:** This application is built for educational, research, and technical evaluation purposes only. It is not approved for diagnostic clinical use. All diagnostic assessments must be verified by a licensed clinical radiologist.